

MOS Gas Mixer

Modbus RTU Communication Protocol Guide

Developer Manual for Mixing Board (Slave 2) and Omron E5CC Temperature Controller (Slave 1)

Document Version:	2.0 (English)
Target Firmware:	TronV41.hex (Updated: June 2026)
Date of Release:	June 07, 2026
Classification:	Technical Integration Manual

Table of Contents

Section	Description	Page
1. Communication Settings	Hardware-level interface parameters and core Modbus function codes.	3
2. Mixing Board Registers	Modbus register mapping for SCCM Configuration, Control, and actual measurements.	4
3. Omron E5CC Controller	Temperature controller parameters, scaling factor, and Status bit breakdown.	6
4. Modbus RTU Byte Frames	Annotated, byte-level raw request and response examples.	8
5. Developer Python Code	Complete Python code reference using <i>pymodbus</i> library.	10

1. Communication Settings

This guide outlines the serial communication protocol settings and register definitions required to interface with the gas mixing system. Communication is conducted via **Modbus RTU** protocol over a serial line.

1.1. Physical Layer Configuration

Parameter	Value	Note
Interface	Serial RS-232 / RS-485	Differential or single-ended serial communication
Protocol	Modbus RTU	Standard Modbus RTU protocol structure
Baud Rate	19,200 bps	Fixed baudrate for both modules
Data Bits	8 bits	Standard Modbus character size
Parity	Even (E)	Parity bit check enabled
Stop Bits	1 bit	Total frame configuration: 8E1
CRC Check	CRC-16	Standard Modbus CRC-16 (Polynomial: 0x8005)
Suggested Timeout	≥ 100 ms (Mixing Board) ≥ 200 ms (Omron E5CC)	Recommended master poll timeout for reliable communications
Slave Address	0x02 (2): Mixing Board 0x01 (1): Omron E5CC	Module designations on the shared serial bus

1.2. Supported Modbus Function Codes

The system responds to four core Modbus function codes (FC). Note that attempting to write registers using unsupported codes or out-of-range addresses will return an error code.

FC (Hex)	FC (Dec)	Function Name	Transmission Direction	Intended Purpose
0x03	03	Read Holding Registers	Master → Slave → Master	Reads SCCM hardware configuration, setpoints, DAC status, and E5CC controller variables.
0x04	04	Read Input Registers	Master → Slave → Master	Reads real-time actual flow (SCCM) values and Status flags.
0x06	06	Write Single Register	Master → Slave	Controls relay switches (ON/OFF), enables/disables DAC output, and sends E5CC operation commands.
0x10	16	Write Multiple Registers	Master → Slave	Writes 32-bit floating-point registers (occupies 2 registers) for setpoints and sensor configurations.

2. Mixing Board Registers (Slave 0x02)

The Mixing Board supports 6 channels of Mass Flow Controllers (SCCM). Floating-point values (such as flow rates and voltages) are encoded in standard **32-bit IEEE 754 float** format. Since each Modbus register holds 16 bits, a float value spans **two consecutive registers**.

CRITICAL NOTE ON ENDIANNESS: Float values use a **Big-Endian Word** layout. This means the *High Word* (containing the sign and exponent bits) is stored at the lower address, and the *Low Word* is stored at the higher address.
Python decoding: `value = struct.unpack('>f', struct.pack('>HH', high_word, low_word))[0]`

2.1. SCCM Hardware Configuration Registers (Holding 0 – 47)

These registers define the calibration configuration for the 6 SCCM channels. Each channel uses 8 consecutive registers (4 float variables) representing the minimum flow, maximum flow, minimum voltage, and maximum voltage. Readings are performed using FC 0x03, and writes using FC 0x10.

Dec Addr	Hex Addr	Channel	Parameter	Data Type	Unit	Default	Description
0	0x0000	SCCM 1	min-sccm [High Word] min-sccm [Low Word]	uint16 uint16	SCC M	0.0	Minimum measurable flow sensor limit (combined into IEEE 754 float)
2	0x0002		max-sccm [High Word] max-sccm [Low Word]	uint16 uint16	SCC M	500.0	Maximum measurable flow sensor limit (combined into IEEE 754 float)
4	0x0004		min-voltage [High Word] min-voltage [Low Word]	uint16 uint16	V	0.0	Output voltage at minimum flow rate (combined into IEEE 754 float)
6	0x0006		max-voltage [High Word] max-voltage [Low Word]	uint16 uint16	V	5.0	Output voltage at maximum flow rate (combined into IEEE 754 float)
8	0x0008	SCCM 2	min-sccm [High Word] min-sccm [Low Word]	uint16 uint16	SCC M	0.0	Minimum measurable flow sensor limit (combined into IEEE 754 float)
10	0x000A		max-sccm [High Word] max-sccm [Low Word]	uint16 uint16	SCC M	500.0	Maximum measurable flow sensor limit (combined into IEEE 754 float)
12	0x000C		min-voltage [High Word] min-voltage [Low Word]	uint16 uint16	V	0.0	Output voltage at minimum flow rate (combined into IEEE 754 float)
14	0x000E		max-voltage [High Word] max-voltage [Low Word]	uint16 uint16	V	5.0	Output voltage at maximum flow rate (combined into IEEE 754 float)

16	0x0010	SCCM 3	min-sccm [High Word] min-sccm [Low Word]	uint16 uint16	SCCM	0.0	Minimum measurable flow sensor limit (combined into IEEE 754 float)
18	0x0012		max-sccm [High Word] max-sccm [Low Word]	uint16 uint16	SCCM	500.0	Maximum measurable flow sensor limit (combined into IEEE 754 float)
20	0x0014		min-voltage [High Word] min-voltage [Low Word]	uint16 uint16	V	0.0	Output voltage at minimum flow rate (combined into IEEE 754 float)
22	0x0016		max-voltage [High Word] max-voltage [Low Word]	uint16 uint16	V	5.0	Output voltage at maximum flow rate (combined into IEEE 754 float)
24	0x0018	SCCM 4	min-sccm [High Word] min-sccm [Low Word]	uint16 uint16	SCCM	0.0	Minimum measurable flow sensor limit (combined into IEEE 754 float)
26	0x001A		max-sccm [High Word] max-sccm [Low Word]	uint16 uint16	SCCM	500.0	Maximum measurable flow sensor limit (combined into IEEE 754 float)
28	0x001C		min-voltage [High Word] min-voltage [Low Word]	uint16 uint16	V	0.0	Output voltage at minimum flow rate (combined into IEEE 754 float)
30	0x001E		max-voltage [High Word] max-voltage [Low Word]	uint16 uint16	V	5.0	Output voltage at maximum flow rate (combined into IEEE 754 float)
...	...	SCCM 3 - 6	Repeated Pattern	uint16	—	—	Pattern repeats similarly: Ch {n} starts at address (n - 1) * 8

2.2. Control Registers: Relays & Setpoints (Holding 20-21, 60-77)

Control functions are separated into discrete Relay operations (for valves/power lines) and SCCM Channel controls (combining Setpoints and DAC status indicators).

Relay Control Addresses (Holding 20 - 21)

Dec Addr	Hex Addr	Parameter	Data Type	Value = 0	Value = 1	Write FC	Read FC	Description
20	0x0014	Relay 1	uint16	OFF	ON	0x06	0x03	Write 0x0001 to activate relay, 0x0000 to deactivate.
21	0x0015	Relay 2	uint16	OFF	ON	0x06	0x03	Write 0x0001 to activate relay, 0x0000 to deactivate.

Setpoint & DAC Control (Holding 60 - 77)

Each SCCM channel contains a 3-register control sequence: [Setpoint_Hi] [Setpoint_Lo] [DAC_Enable]. The setpoint must be written as a 32-bit float across 2 registers using FC 0x10. The DAC_Enable register is handled as a single register via FC 0x06.

Dec Addr	Hex Addr	Channel	Register / Purpose	Data Type	Write Value	FC	Description
60	0x003C	SCCM 1	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
61	0x003D		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word
62	0x003E		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel
63	0x003F	SCCM 2	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
64	0x0040		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word
65	0x0041		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel
66	0x0042	SCCM 3	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
67	0x0043		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word
68	0x0044		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel
69	0x0045	SCCM 4	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
70	0x0046		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word
71	0x0047		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel
72	0x0048	SCCM 5	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
73	0x0049		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word

74	0x004A		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel
75	0x004B	SCCM 6	Setpoint [High Word]	uint16	Float (Part 1)	0x10	Flow setpoint high word
76	0x004C		Setpoint [Low Word]	uint16	Float (Part 2)	0x10	Flow setpoint low word
77	0x004D		DAC Enable	uint16	0 = OFF 1 = ON	0x06	Enables analogue DAC output for the channel

2.3. Real-Time Measured Values (Input Registers 0 – 12)

Input registers represent hardware measurements and are read-only (FC 0x04). The flow values are updated in real-time as 32-bit floats. Address 12 represents status flag data.

Dec Addr	Hex Addr	Channel	Register / Parameter	Data Type	Unit	Range	Description
0	0x0000	SCCM 1	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
1	0x0001		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
2	0x0002	SCCM 2	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
3	0x0003		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
4	0x0004	SCCM 3	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
5	0x0005		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
6	0x0006	SCCM 4	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
7	0x0007		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
8	0x0008	SCCM 5	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
9	0x0009		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
10	0x000A	SCCM 6	PV [High Word]	uint16	SCCM	0.0 - Max	Flow rate reading high word
11	0x000B		PV [Low Word]	uint16	SCCM	—	Flow rate reading low word
12	0x000C	—	Status / Flag	uint16	—	0 or 1	General module state variable

3. Omron E5CC Temperature Controller Registers

The system incorporates an Omron E5CC temperature controller addressed as **Slave 1 (0x01)**. Unlike the Mixing Board, which utilizes 32-bit floats, the E5CC uses signed and unsigned **16-bit integers**. To represent fractional numbers, many values are scaled by a factor of 10.

IMPORTANT VALUE SCALING: Standard temperature, setpoint, and alarm limits are transmitted as integer values scaled by **×0.1**. For example, a physical temperature value of **25.0 °C** is transmitted as the integer **250**.

To read: `actual_value = register_value / 10.0`

To write: `register_value = int(actual_value * 10)`

3.1. E5CC Register Protocol Mapping

Hex Addr	Dec Addr	Type	R/W	Variable Name / Function	Scaling	FC	Notes / Ranges
0x0000	0	Holding	R/W	Run / Stop Command	—	0x06	0 = Run, 1 = Stop
0x0002	2	Holding	R/W	Auto-Tune (AT) Execute	—	0x06	0 = Stop, 1 = Start AT
0x0100	256	Holding	R	Status Monitor Flags	Bit field	0x03	See Status Bits breakdown below
0x2000	8192	Holding	R	Process Value (PV)	×0.1 °C	0x03	Current temperature reading
0x2001	8193	Holding	R	Setpoint (SP) Monitor	×0.1 °C	0x03	Active effective setpoint
0x2002	8194	Holding	R	Output MV Monitor	×0.1 %	0x03	Current control output (0.0 to 100.0%)
0x2100	8448	Holding	R/W	Set Point 1 (SP1)	×0.1 °C	0x06/0x10	Primary target temperature
0x2101	8449	Holding	R/W	Set Point 2 (SP2)	×0.1 °C	0x06/0x10	Secondary target setpoint
0x2200	8704	Holding	R/W	Alarm Value 1	×0.1 °C	0x06	Alarm threshold limit 1
0x2201	8705	Holding	R/W	Alarm Value 2	×0.1 °C	0x06	Alarm threshold limit 2
0x2300	8960	Holding	R/W	Proportional Band (P)	×0.1	0x06	PID proportional term
0x2301	8961	Holding	R/W	Integral Time (I)	1 sec	0x06	PID integral term in seconds
0x2302	8962	Holding	R/W	Derivative Time (D)	×0.1 sec	0x06	PID derivative term
0x2303	8963	Holding	R/W	Control Period	1 sec	0x06	Output switching rate (Relay output)
0x2304	8964	Holding	R/W	MV Upper Limit	×0.1 %	0x06	Maximum control output ceiling
0x2305	8965	Holding	R/W	MV Lower Limit	×0.1 %	0x06	Minimum control output floor
0x2400	9216	Holding	R/W	Input Shift (Offset)	×0.1 °C	0x06	Sensor calibration correction value
0x2401	9217	Holding	R/W	SP Upper Limit	×0.1 °C	0x06	Maximum setpoint boundary
0x2402	9218	Holding	R/W	SP Lower Limit	×0.1 °C	0x06	Minimum setpoint boundary
0x2500	9472	Holding	R/W	Manual MV	×0.1 %	0x06	Manual control value (active in Manual Mode)

3.2. Status Monitor Bit Flags (Holding Address 0x0100)

Reading register 0x0100 returns a 16-bit status mask. Code implementations should perform bitwise operation logic to decode the status variables:

Bit Position	Variable Name	State (Bit = 1)	State (Bit = 0)	Description
Bit 0	RUN / STOP	STOP (d l ing)	RUN (l l ang ch l ly)	Controller execution state
Bit 1	Auto / Manual	Manual mode	Auto mode	Feedback loop mode control
Bit 2	Auto-Tune (AT)	AT active / running	AT inactive	PID self-calibration state
Bit 3	Alarm 1	Alarm 1 triggered	Normal	Alarm threshold 1 flag
Bit 4	Alarm 2	Alarm 2 triggered	Normal	Alarm threshold 2 flag
Bit 5	Alarm 3	Alarm 3 triggered	Normal	Alarm threshold 3 flag
Bit 6	Heater Break (HB)	Heater wire broken	Normal	Sensor detects load cut (if hardware exists)
Bit 7	Input Error	Sensor failure / Overflow	Normal	Temperature probe failure warning
Bit 8	Output 1	Control Output 1 is ON	Output 1 is OFF	Physical output relay/SSR pulse state
Bit 9	Output 2	Control Output 2 is ON	Output 2 is OFF	Sub-output relay pulse state
Bit 10	SP Selection	Setpoint 2 active	Setpoint 1 active	Displays which setpoint channel is active
Bit 11	SP3 / SP4 Selection	Setpoint 3 active	SP1 / SP2 active	Additional active setpoint channels

4. Modbus RTU Frame Examples (Byte Level)

Below are documented byte arrays for common requests and responses. All frames are hexadecimal. The CRC-16 checksums are calculated using the standard Modbus algorithm (Polynomial: 0x8005).

4.1. Mixing Board: Read All Actual SCCM Flow Values (Slave 2)

Reads 12 input registers (representing 6 float values) starting at address 0x0000. Function code used is **0x04 (Read Input Registers)**.

```

REQUEST  --> [02] [04] [00] [00] [00] [0C] [71] [CB]
              |  |  |  |  |  |  |  |
              |  |  |  |  |  |  |  +-- CRC-16 (Low, High)
              |  |  |  |  |  |  |  +----- Quantity: 12 registers (0x000C)
              |  |  |  |  |  |  |  +----- Start Address: 0x0000
              |  |  |  |  |  |  |  +----- Function Code: 0x04
              |  |  |  |  |  |  |  +----- Slave Address: 0x02
              +-----

RESPONSE <-- [02] [04] [18] [42] [48] [00] [00] ... [XX] [XX]
              |  |  |  |  |  |  |  |
              |  |  |  |  |  |  |  +-- CRC-16
              |  |  |  |  |  |  |  +----- SCCM 1 value: 0x42480000 (50.0 SCCM)
              |  |  |  |  |  |  |  +----- Byte count: 24 bytes (12 registers * 2)
              |  |  |  |  |  |  |  +----- Function Code: 0x04
              |  |  |  |  |  |  |  +----- Slave Address: 0x02
              +-----

```

4.2. Mixing Board: Write Setpoint SCCM 1 = 50.0 (Slave 2)

Writes the 32-bit float value 50.0 (IEEE 754 float: 0x42480000) to Holding registers 60 and 61. Function code used is **0x10 (Write Multiple Registers)**.

```

REQUEST  --> [02] [10] [00] [3C] [00] [02] [04] [42] [48] [00] [00] [A9] [C0]
              |  |  |  |  |  |  |  |  |  |  |  |  |  |
              |  |  |  |  |  |  |  |  |  |  |  |  |  +-- CRC-16
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Data -----
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Byte Count: 4 bytes
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Quantity: 2 registers
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Start Address: 0x003C (60)
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Function Code: 0x10
              |  |  |  |  |  |  |  |  |  |  |  |  |  +----- Slave Address: 0x02
              +-----

RESPONSE <-- [02] [10] [00] [3C] [00] [02] [81] [D5]
              |  |  |  |  |  |  |  |
              |  |  |  |  |  |  |  +-- CRC-16
              |  |  |  |  |  |  |  +----- Quantity: 2 registers written
              |  |  |  |  |  |  |  +----- Start Address: 0x003C (60)
              |  |  |  |  |  |  |  +----- Function Code: 0x10
              |  |  |  |  |  |  |  +----- Slave Address: 0x02
              +-----

```

4.3. Mixing Board: Enable DAC Channel 1 (Slave 2)

Enables analog output by writing 0x0001 to register 62 (0x003E) using **FC 0x06 (Write Single Register)**.

```

REQUEST  --> [02] [06] [00] [3E] [00] [01] [69] [CA]
              |  |  |  |  |  |  |  |
              |  |  |  |  |  |  |  +-- CRC-16
              |  |  |  |  |  |  |  +----- Value: Enable = 1 (0x0001)
              |  |  |  |  |  |  |  +----- Address: 0x003E (62)
              |  |  |  |  |  |  |  +----- Function Code: 0x06
              |  |  |  |  |  |  |  +----- Slave Address: 0x02
              +-----

RESPONSE <-- [02] [06] [00] [3E] [00] [01] [69] [CA] (Echo of Request)

```

4.4. E5CC: Read Process Value / Temperature (Slave 1)

Reads the current temperature (PV) at holding register 0x2000 using **FC 0x03 (Read Holding Registers)**.

```

REQUEST  -->  [01] [03] [20] [00] [00] [01] [8E] [3A]
               |  |  |  |  |  |  +--- CRC-16
               |  |  |  |  |  +----- Quantity: 1 register
               |  |  |  |  +----- Start Address: 0x2000
               |  |  |  +----- Function Code: 0x03
               |  +----- Slave Address: 0x01
               +-----

RESPONSE <-- [01] [03] [02] [00] [FA] [B8] [64]
               |  |  |  |  |  |  +--- CRC-16
               |  |  |  |  |  +----- PV Data: 0x00FA = 250 (25.0 °C)
               |  |  |  |  +----- Byte count: 2 bytes
               |  |  |  +----- Function Code: 0x03
               |  +----- Slave Address: 0x01
               +-----

```

4.5. E5CC: Write Target Setpoint = 25.0 °C (Slave 1)

Writes the target setpoint SP1 (register 0x2100) as the scaled value 250 (0x00FA) using **FC 0x06 (Write Single Register)**.

```

REQUEST  -->  [01] [06] [21] [00] [00] [FA] [89] [B1]
               |  |  |  |  |  |  +--- CRC-16
               |  |  |  |  |  +----- Value to Write: 250 (0x00FA)
               |  |  |  |  +----- Address: 0x2100
               |  |  |  +----- Function Code: 0x06
               |  +----- Slave Address: 0x01
               +-----

RESPONSE <-- [01] [06] [21] [00] [00] [FA] [89] [B1] (Echo of Request)

```

4.6. E5CC: Run / Stop Commands (Slave 1)

Sends operation status controls to register 0x0000 via **FC 0x06 (Write Single Register)**. Write 0x0000 to RUN the controller, or 0x0001 to STOP it.

```

RUN COMMAND:
REQUEST  -->  [01] [06] [00] [00] [00] [00] [89] [CA]
               |  |  |  |  |  |  +--- CRC-16
               |  |  |  |  |  +----- Value: 0 = RUN
               |  |  |  |  +----- Address: 0x0000
               |  |  |  +----- Function Code: 0x06
               |  +----- Slave Address: 0x01
               +-----

STOP COMMAND:
REQUEST  -->  [01] [06] [00] [00] [00] [01] [48] [0A] (Value: 1 = STOP)

```

5. Developer Python Code Examples

This section contains complete Python implementation templates for developers, using the popular **pymodbus** library. Ensure the package is installed using `pip install pymodbus`.

5.1. Mixing Board Integration Code Reference (Slave 2)

```
import struct
from pymodbus.client import ModbusSerialClient

# 1. Initialize Modbus RTU Client
client = ModbusSerialClient(
    method='rtu',
    port='COM3',          # Adjust serial port interface identifier
    baudrate=19200,
    bytesize=8,
    parity='E',           # Even parity check (8E1)
    stopbits=1,
    timeout=0.5
)
client.connect()
SLAVE = 2                # Mixing Board Slave ID

# 2. Helper functions for encoding and decoding floats
def regs_to_float(hi, lo):
    # Reconstruct IEEE 754 float from high/low 16-bit register words
    return struct.unpack('>f', struct.pack('>HH', hi, lo))[0]

def float_to_regs(value):
    # Convert float into high/low 16-bit registers
    hi, lo = struct.unpack('>HH', struct.pack('>f', value))
    return hi, lo

# 3. Read Flow Measurements from Input Registers 0-11
def read_sccm_all():
    r = client.read_input_registers(0, 12, slave=SLAVE) # FC 0x04
    if r.isError():
        raise RuntimeError("Failed to read SCCM inputs")
    regs = r.registers
    result = {}
    for ch in range(6):
        hi = regs[ch * 2]
        lo = regs[ch * 2 + 1]
        result[f'SCCM_{ch+1}'] = regs_to_float(hi, lo)
    return result

# 4. Write Flow Setpoint (FC 0x10) and Enable DAC (FC 0x06)
def set_sccm_setpoint(channel, value):
    # Channel ranges: 1 to 6
    base = 60 + (channel - 1) * 3
    hi, lo = float_to_regs(value)
    client.write_registers(base, [hi, lo], slave=SLAVE) # FC 0x10

def set_dac_enable(channel, enable):
    base = 60 + (channel - 1) * 3
    dac_reg = base + 2
    val = 1 if enable else 0
    client.write_register(dac_reg, val, slave=SLAVE)    # FC 0x06

# 5. Control Relay Switches (FC 0x06)
def set_relay(relay_num, state):
    # relay_num: 1 or 2
    addr = 19 + relay_num
    val = 1 if state else 0
    client.write_register(addr, val, slave=SLAVE)      # FC 0x06
```

5.2. Omron E5CC Temperature Controller Integration Code (Slave 1)

```
import time
from pymodbus.client import ModbusSerialClient

client = ModbusSerialClient(
    method='rtu',
    port='COM4',
    baudrate=19200,
    bytesize=8,
    parity='E',
    stopbits=1,
    timeout=0.5
)
client.connect()
SLAVE = 1          # Omron E5CC Slave ID

# 1. Read Operations (using FC 0x03)
def read_PV():
    r = client.read_holding_registers(0x2000, 1, slave=SLAVE)
    return r.registers[0] / 10.0 # Apply 1/10 scaling factor for °C

def read_SP():
    r = client.read_holding_registers(0x2001, 1, slave=SLAVE)
    return r.registers[0] / 10.0

def read_status():
    r = client.read_holding_registers(0x0100, 1, slave=SLAVE)
    status = r.registers[0]
    return {
        'stop':    bool(status & (1 << 0)), # Bit 0 = STOP
        'manual':  bool(status & (1 << 1)), # Bit 1 = Manual
        'at_run':  bool(status & (1 << 2)), # Bit 2 = Auto-Tune active
        'alarm1':  bool(status & (1 << 3)), # Bit 3 = Alarm 1
        'alarm2':  bool(status & (1 << 4)), # Bit 4 = Alarm 2
        'error':   bool(status & (1 << 7)), # Bit 7 = Input Error
    }

# 2. Control & Write Operations (using FC 0x06)
def set_SP(temp_celsius):
    # Scaled by 10 before writing
    val = int(temp_celsius * 10)
    client.write_register(0x2100, val, slave=SLAVE)

def run_controller():
    client.write_register(0x0000, 0, slave=SLAVE) # Write 0x0000

def stop_controller():
    client.write_register(0x0000, 1, slave=SLAVE) # Write 0x0001

def start_auto_tune():
    client.write_register(0x0002, 1, slave=SLAVE) # Write 0x0001 to start

def stop_auto_tune():
    client.write_register(0x0002, 0, slave=SLAVE) # Write 0x0000 to stop

# 3. Calibration Configuration Settings
def set_PID(P, I, D):
    client.write_register(0x2300, int(P * 10), slave=SLAVE) # Proportional
    client.write_register(0x2301, int(I), slave=SLAVE)     # Integral (seconds)
    client.write_register(0x2302, int(D * 10), slave=SLAVE) # Derivative

def set_SP_limits(sp_min, sp_max):
    client.write_register(0x2402, int(sp_min * 10), slave=SLAVE) # Lower limit
    client.write_register(0x2401, int(sp_max * 10), slave=SLAVE) # Upper limit
```